SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
(Common to all Branches)
SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu

Mode of Exam
OFFLINE

Academic Year: 2023-24 (ODD)

RegNo

Test: CLAT-1 **Date: 18-08-2023**
Course Code & Title: 21CSS201T COA **Duration: 100 minutes**
Year & Sem: II & III **SET A** **Max. Marks: 50**

Course Articulation Matrix:

Course Learning Outcomes (CLO)	At the end of this course, learners will be able to:	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12
CO-1	Identify the computer hardware and how software interacts with computer hardware	M	H	-		-	-	-	-	-	-	-	-
CO-2	Apply Boolean algebra as related to designing computer logic, through simple combinational and sequential logic circuits	L	H	-		-	-	-	-	-	-	-	-

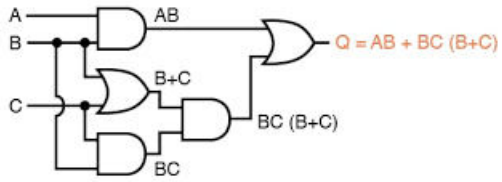
Part - A
(1 x 10 = 10 Marks)

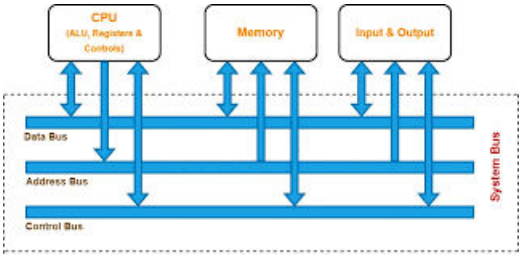
Instructions: Answer all

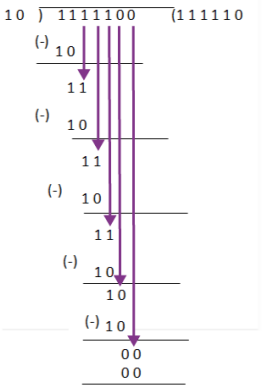
Q. No	Question	Marks	BL	CO	PO	PI Code
1	**Convert the given decimal number 245 to hexadecimal** _____ **a) $(OxF5)_{16}$** b) $(OxE5)_{16}$ c) $(OxFF)_{16}$ d) $(Ox2F)_{16}$	1	3	1	2	2.1.2
2	**Which type of computer is typically used by large organizations for data processing, business applications, and centralized computing?** a) Minicomputer b) Personal Computer **c) Mainframe Computer** d) Tablet Computer	1	3	1	2	2.1.2
3	**Convert the excess-3 code 0110 to Gray code.** a) 0011 b) 0101 c) 1101 d) 1011	1	3	1	2	2.1.2
4	**Which of the following is true about the 2's complement representation?** a) The leftmost bit is the sign bit. b) Positive numbers are represented by a leading 0. **c) Negative numbers are represented by a leading 1.** d) Positive numbers have a negative counterpart. 0101.	1	3	1	2	2.1.2
5	**Which logic gate produces an output of 0 only when all of its inputs are 1?** a) AND gate	1	2	1	2	2.1.2

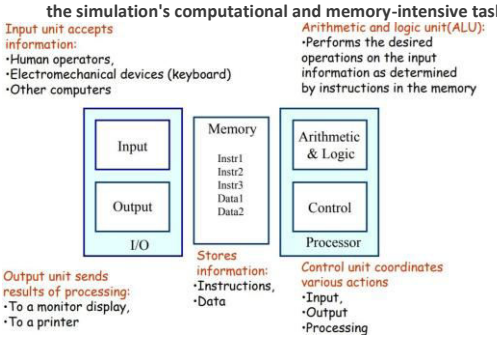
	b) OR gate **c) NAND gate** **d) NOR gate**					
6	**Which integer representation is commonly used in digital computers for arithmetic operations?** a) Roman numeral b) Binary-coded decimal (BCD) c) Excess-3 **d) Two's complement**	**1**	**3**	**1**	**2**	**2.1.2**
7	**How are negative numbers represented in sign-magnitude?** a) By setting all bits to 1 **b) By setting the sign bit to 1 and the magnitude bits to the binary representation of the absolute value** c) By setting the sign bit to 0 and the magnitude bits to the binary representation of the absolute value d) By setting the sign bit to 1 and the magnitude bits to the one's complement of the absolute value	**1**	**3**	**1**	**2**	**2.1.2**
8	**In BCD addition, if the sum of two BCD digits is greater than 9, what additional step is necessary?** a) Subtract 10 **b) Carry over to the next digit** c) Convert to binary d) Round up	**1**	**3**	**1**	**2**	**2.1.2**
9	**What is the minimum number of bits required to address 1024 distinct memory locations?** a) 6 bits **b) 10 bits** c) 16 bits d) 32 bits	**1**	**2**	**1**	**2**	**2.1.2**
10	**In a little-endian memory system, where is the least significant byte (LSB) of a multi-byte data stored?** a) At the highest memory address **b) At the lowest memory address** c) In the middle of the memory d) In a separate memory location	**1**	**3**	**1**	**2**	**2.1.2**
Part – B (4*4 = 16 Marks) Answer any four questions						
11	**signed number subtraction using two's complement representation.** Example 1: Subtract: 6 - 3 Step 1: Convert both numbers to binary (using 4-bit representation): 6 = 0110	**4**	**3**	**1**	**2**	**2.1.2**

	3 = 0011 Step 2: Find the two's complement of the second number (subtrahend): 1's complement of 0011 = 1100 Add 1 to get the two's complement: 1100 + 1 = 1101 Step 3: Add the first number (minuend) and the negative of the second number (two's complement of subtrahend): 0110 + 1101 Subtract: -8 - (-3) Step 1: Convert both numbers to binary (using 4-bit representation): -8 = 1000 (2's complement form) -3 = 1101 (2's complement form) Step 2: Add the two numbers (remember, subtracting a negative is equivalent to adding a positive): 1000 + 1101 1 0101 Discard the carry, and the result is 0101. Step 3: Convert the binary result back to decimal: 0101 = 5 So, -8 - (-3) = 5. Example 2: Subtract: 9 - (-6) Step 1: Convert both numbers to binary (using 4-bit representation): 9 = 1001 -6 = 1010 (2's complement form) Step 2: Add the two numbers (subtracting a negative is like adding a positive): 1001 + 1010 1 0011 Discard the carry, and the result is 0011. Step 3: Convert the binary result back to decimal: 0011 = 3 So, 9 - (-6) = 3. Example 3: Subtract: -15 - 3 Step 1: Convert both numbers to binary (using 5-bit representation): -15 = 10001 (2's complement form)					

	3 = 00011 Step 2: Add the two numbers (subtracting a positive is like adding a negative): 10001 + 00011 1 01000 Discard the carry, and the result is 01000. Step 3: Convert the binary result back to decimal: 01000 = 8 So, -15 - 3 = -8.					
12	**Draw the circuit diagram for the following logic expression AB + BC(B + C)**  	**4**	**3**	**1**	**2**	**2.1.2**
13	**Convert the BCD number 725 to Gray code.** Step 1: Convert BCD to binary: 7 -> 0111 2 -> 0010 5 -> 0101 Step 2: Generate Gray code for each binary digit: For 7 (0111), Gray code = 0111 (same as binary). For 2 (0010), Gray code = XOR(0, 0) XOR(0, 1) XOR(1, 0) XOR(0, 0) = 0011. For 5 (0101), Gray code = XOR(0, 1) XOR(1, 0) XOR(0, 1) XOR(1, 0) = 1110. So, the Gray code representation for the BCD number 725 is 0111 0011 1110.	**4**	**3**	**1**	**2**	**2.1.2**
14	**Convert decimal $(42)_{10}$ to Excess-3.** Excess-3 representation is a binary-coded decimal (BCD) variant in which each decimal digit is represented by its corresponding 4-bit binary value plus 0011 (decimal 3). To convert a decimal number to Excess-3, follow these steps: Convert the decimal number to its BCD representation. Add 3 (0011 in binary) to each BCD digit. Let's convert the decimal number 42 to Excess-3: Step 1: Convert decimal 42 to BCD:	**4**	**3**	**1**	**2**	**2.1.2**

	4 -> 0100 2 -> 0010 Step 2: Add 3 (0011) to each BCD digit: For 4 (0100), Excess-3 = 0100 + 0011 = 0111 For 2 (0010), Excess-3 = 0010 + 0011 = 0101 So, the Excess-3 representation of decimal 42 is 0111 0101.					
15	**Write in detail about the Bus structure in COA** Buses play a crucial role in enabling data, control signals, and addresses to flow between the central processing unit (CPU), memory, input/output devices, and other system components. The concept of bus structure is a fundamental aspect of computer organization and architecture (COA). Let's delve deeper into the types and functions of buses within a computer system:  **Data Bus:** The data bus is responsible for carrying data between the CPU, memory, and I/O devices. It is bidirectional, allowing data to be read from memory or devices and written to them. The width of the data bus determines the number of bits that can be transferred simultaneously. Wider data buses result in higher data transfer rates. Address Bus: The address bus carries the memory address of the data being transferred. The CPU uses the address bus to specify the location in memory or an I/O device to read from or write to. The width of the address bus determines the maximum memory address that can be accessed, which directly affects the addressable memory space. Control Bus: The control bus carries control signals that coordinate the activities of various components within the computer system. Control signals include read and write signals, interrupt requests, clock signals, and other signals that help manage data transfer and synchronization.	**4**	**1**	**1**	**1**	**1.3.1**
	Part – C **(2*12 = 24 Marks)**					
20	Perform the following conversion (i) You are a computer systems engineer working on a project that involves converting numbers between different bases. One of your tasks is to convert the octal number (176.35)8 to its binary equivalent. **octal number (176.35)_8 is equal to (001111110.011101)_2 in binary** (ii) Perform the binary to decimal conversion for the given number, $(1011100011001.01)_2$. Describe the steps you take to convert the binary fractional part to its decimal equivalent, and explain how you combine the integer and fractional parts to obtain the final decimal result. **binary number (1011100011001.01)_2 is equal to 7018.25 in decimal.** (iii) You are a systems hardware engineer working on a project that involves converting numbers between different bases. One of your tasks is to convert the decimal number (7892) 10 to hexadecimal equivalent. **decimal number 7892 is equal to 1E4 in hexadecimal.**	**12**	**3**	**1**	**2**	**2.1.2**

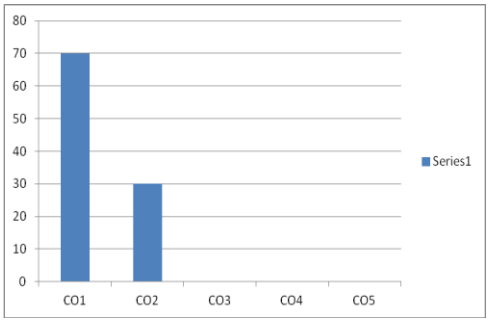
	iv) Perform BCD addition on the following numbers 652.21 and 345.12 **Ans : 0110 0101 0010.0010 0001** **0011 0100 0101.0001 0010** **(OR)** **b. i) A student approaches you with a question about subtracting (-19) from (23) using two's complement notation. Explain the step-by-step process of performing this subtraction using the two's complement method. Show the necessary binary conversions, including finding the two's complement of the negative number, and then walk through the binary subtraction and any necessary carry operations. Finally, interpret the binary result back to decimal.** Convert to Binary: Convert both numbers (-19 and 23) to their 2's complement binary representations. -19 in binary (8-bit representation): 11101101 23 in binary (8-bit representation): 00010111 Find 2's Complement of Subtrahend (23): Invert all the bits in the binary representation of 23: 00010111 becomes 11101000 Add 1 to the inverted value: 11101000 + 1 = 11101001 Perform Addition: Now, add the 2's complement of 23 to -19. 11101101 (-19) + 11101001 (2's complement of 23) --- 110101110 Overflow Check: Since we're using an 8-bit representation, the result should be interpreted as 101110. The most significant bit (MSB) is 1, indicating a negative value. Convert Back to Decimal: Convert the binary result 101110 back to decimal. Invert the bits and add 1: 010001 + 1 = 17 Since the MSB is 1, the result is -17. ii). **Imagine you're designing a digital circuit that needs to multiply two binary numbers: 01101 and 11010. Perform the binary multiplication step by step, showing all intermediate products and the final result.** 01101 (13 in decimal) × 10101 (21 in decimal) --- 01101 (Partial product, shifted 0 positions) 01101 (Partial product, shifted 2 positions) + 01101 (Partial product, shifted 4 positions) + 00000 (Partial product, shifted 6 positions) --- 11100101 iii). **Using binary division, compute the quotient and remainder when dividing 01111100_2 by 0010_2. Begin by converting the numbers to binary representation and then demonstrate the step-by-step process of binary division. Highlight the key steps involved in each iteration of the division process, including how you determine the next quotient bit and how you adjust the remainder. Conclude by providing the final quotient and remainder in binary and decimal** formats.  					
21	**a. i) Convert excess-3 1001 to binary.** Convert each digit in the excess-3 code to its corresponding decimal value by subtracting 3: 1 (excess-3) = 1 - 3 = -2 0 (excess-3) = 0 - 3 = -3 0 (excess-3) = 0 - 3 = -3 1 (excess-3) = 1 - 3 = -2	**12**	**3**	**1**	**2**	**2.1.2**

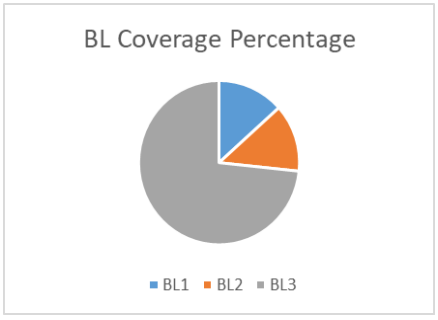
	Convert the resulting decimal values to their 4-bit binary representations: -2 in decimal = 1110 in binary (2's complement of 2) -3 in decimal = 1101 in binary (2's complement of 3) Concatenate the binary representations together to get the binary representation of the excess-3 code 1001: (1001)_excess-3 = (1110 1101)_binary So, the excess-3 code 1001 is equal to (1110 1101)_binary in binary. **ii) Convert decimal 0 to excess-3 code.** Add 3 to the decimal number: 0 + 3 = 3 Convert the result (3) to its binary representation. Since it's a single-digit decimal, the Excess-3 code will be a 4-bit code: 3 in binary: 0011 So, the decimal number 0 is represented as 0011 in Excess-3 code. **iii) Convert binary 1111 to excess-3 code** Add 3 to each decimal digit of the binary number: 1 + 3 = 4 1 + 3 = 4 1 + 3 = 4 1 + 3 = 4 Convert the resulting decimal digits to their 4-bit binary representations: 4 in decimal = 0100 in binary 4 in decimal = 0100 in binary 4 in decimal = 0100 in binary 4 in decimal = 0100 in binary Concatenate the binary representations together to get the excess-3 code: (1111)_binary = (0100 0100 0100 0100)_excess-3 So, the binary number 1111 is equal to (0100 0100 0100 0100)_excess-3 in excess-3 code. **(OR)** **b. Describe how the different functional units of the CPUs collaborate to execute this simulation effectively. Explain the roles of the integer unit, floating-point unit, memory unit, and control unit in processing the simulation's computational and memory-intensive tasks.**  Input Unit: The input unit is responsible for receiving and decoding external input data that serves as the basis for the simulation. This can include parameters, initial conditions, or configuration settings required for the simulation. The input unit interfaces with external devices or sources, such as sensors or user inputs, and converts the incoming data into a format that the CPU and simulation can understand. Output Unit: The output unit handles the presentation and storage of simulation results. Once the simulation computations are complete, the output unit formats the data for display or storage. This may involve converting internal data representations to human-readable formats or saving results in appropriate files or databases. The output unit can interface with displays, write to files, or communicate with other systems for data analysis. Memory Unit: The memory unit manages the storage and retrieval of data and instructions required for the simulation. It includes various levels of cache and the main memory. The memory unit	**12**	**2**	**1**	**1**	**2.1.2**

	plays a critical role in handling the vast amount of data associated with simulations. It stores program instructions, intermediate data, and input/output buffers. The memory unit employs techniques such as caching, prefetching, and memory hierarchy management to optimize data access and reduce memory latency. Control Unit: The control unit orchestrates the execution of instructions and manages the flow of operations within the CPU. It fetches instructions from memory, decodes them, and directs them to the appropriate functional units for execution. In simulation tasks, the control unit ensures proper sequencing of computations, handling branching and conditional execution. It also manages pipelining and speculative execution to maximize CPU utilization and performance. Effective Collaboration: The input unit feeds the initial conditions and parameters to the simulation process, which the control unit uses to manage the simulation's execution flow. Computational tasks are carried out by the CPU's ALU and FPU, managed by the control unit. These tasks involve mathematical operations, logical evaluations, and simulations of complex systems. The memory unit provides data storage for input parameters, intermediate calculations, and output data, enabling seamless data movement between functional units. The output unit takes the final results from memory and formats them for display or storage, providing insights into the simulation's behavior and outcomes.					

***Performance Indicators are available separately for Computer Science and Engineering in AICTE examination reforms policy.**

Course Outcome (CO) and Bloom's level (BL) Coverage in Questions





Approved by the Audit Professor/Course Coordinator